

SPRING BOOT

```
package com.example.thymleaf;

import java.util.Arrays;
import java.util.List;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
public class ThymleafController {

    @GetMapping("/hello")
    public String greeting(Model model){
        model.addAttribute("message", "Hello Java students");
        return "welcome";
    }

    @GetMapping("/students")
    public String listStudents(Model model){

        List<String> students = Arrays.asList("Arun", "Raj", "padma",
"Sahana");

        model.addAttribute("studentList", students);

        return "student-list";
    }

    @GetMapping("/signup")
    public String showForm(){
        return "signup";
    }

    @GetMapping("/greet")
    public String handleForm(String name, Model model) {
        model.addAttribute("greeting", "Hello," +name +"!");
        return "result-page";
    }
}
```

```

@GetMapping("/register")
public String showForm(Model model){
    model.addAttribute("user",new User());
    return "register";
}

@PostMapping("/submit")
public String submitForm(@ModelAttribute User user , Model model){
    model.addAttribute("data", user);
    return "result";
}
}

```

```

package com.example.thymleaf;

public class User {

    private String name;
    private String email;

    public String getName() {
        return name;
    }
    public String getEmail() {
        return email;
    }
    public void setName(String name) {
        this.name = name;
    }
    public void setEmail(String email) {
        this.email = email;
    }
}

```

```

<!doctype html>
<html lang="en">
    <head>
        <meta charset="UTF-8" />
        <meta name="viewport" content="width=device-width, initial-scale=1.0" />

```

```

    <title>Register</title>
  </head>
  <body>
    <form th:object="${user}" th:action="@{/submit}" method="post">
      Name : <input type="text" th:field="*{name}" /> Email :
      <input type="text" th:field="*{email}" />

      <button type="submit">Register</button>
    </form>
  </body>
</html>

```

```

<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Result page</title>
  </head>
  <body>
    <h1 th:text="${greeting}">Result</h1>

    <a href="/signup">Go Back</a>
  </body>
</html>

```

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Result</title>
</head>
<body>

  <p>Name: <span th:text="${data.name}"></span></p>
  <p>Name: <span th:text="${data.email}"></span></p>

</body>
</html>

```

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Signup</title>
</head>
<body>

  <form action="/greet">
    <input type="text" name="name" placeholder="Enter your name">
    <button type="submit">Greet me</button>
  </form>

</body>
</html>
```

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Loop</title>
</head>
<body>
  <h2>Loop</h2>
  <ul>
    <li th:each = "s : ${studentList}" th:text = "${s}">Student Name</li>
  </ul>
</body>
</html>
```

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Greeting Page</title>
  </head>
  <body>
    <h1 th:text="${message}">Default Message</h1>
    <h2>Template engine</h2>
  </body>
</html>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.0</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>
  <groupId>com.example</groupId>
  <artifactId>thymleaf</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>thymleaf</name>
  <description/>
  <url/>
  <licenses>
    <license/>
  </licenses>
  <developers>
    <developer/>
  </developers>
  <scm>
    <connection/>
    <developerConnection/>
    <tag/>
    <url/>
  </scm>
  <properties>
    <java.version>21</java.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-webmvc</artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-thymeleaf-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

```

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-webmvc-test</artifactId>
      <scope>test</scope>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-devtools</artifactId>
      <scope>runtime</scope>
      <optional>true</optional>
    </dependency>

  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>

</project>

```

Theory

- **Thymeleaf basics** → Template engine for rendering dynamic HTML in Spring Boot.
- **@Controller vs @RestController** → @Controller returns views, @RestController returns JSON.
- **Model usage** → Model passes data from controller to Thymeleaf templates.
- **Form binding** → th:object binds a form to a Java object, th:field maps fields.
- **Dynamic rendering** → th:each loops through lists, th:if handles conditions.
- **Validation** → Spring Boot integrates with JSR-303 annotations (@NotEmpty, @Email) for form validation.



Coding Round Questions

- Greeting page → Create a controller method /hello that passes a message to a Thymeleaf template and displays it dynamically.
- Student list → Write a Thymeleaf template that loops through a list of student names passed from the controller and displays them in an unordered list.

- Signup form → Implement a Thymeleaf form that accepts a user's name, submits it to /greet, and displays a personalized greeting on a result page.
- User registration → Create a Thymeleaf form bound to a User object with fields name and email. On form submission, display the entered details on a result page.
- Conditional rendering → Using Thymeleaf, display "Pass" if a student's marks are greater than 40, otherwise display "Fail".
- Form validation → Add validation to a Thymeleaf form so that email must be in proper format and name must not be empty. Show error messages dynamically.

⚡ Machine Coding Round Challenge

Problem: Student Management System with Thymeleaf

Requirements:

1. Create a Student entity with fields: id, name, age, course.
2. Build a Thymeleaf page that lists all students from the database.
3. Add a form to register a new student.
4. Add edit and delete functionality.
5. Use JpaRepository for persistence.
6. Display success/error messages dynamically in Thymeleaf templates.

Hint:

- Controller → @GetMapping("/students"), @PostMapping("/add"), @GetMapping("/edit/{id}"), @PostMapping("/update"), @GetMapping("/delete/{id}").
- Thymeleaf → th:each for listing, th:object + th:field for forms.
- Repository → extends JpaRepository<Student, Integer>.